

PRACTICAL WEEK 6

Task 1: Create a client-server application using Named Pipes (FIFOs). The client sends messages and the Server processes and responds to them. Analyze FIFO behavior when multiple clients communicate simultaneously.

```
srikar@DESKTOP-0NIMS2G: ~/week6

srikar@DESKTOP-0NIMS2G:~$ cd ~/week6
srikar@DESKTOP-0NIMS2G:~/week6$ nano client.c
srikar@DESKTOP-0NIMS2G:~/week6$ gcc client.c -o client
srikar@DESKTOP-0NIMS2G:~/week6$ ./client
Client connected to server.
Enter message:
Server: Server processed:
Enter message:
Server: Server processed:
Enter message: Hello Server
Server: Server processed: Hello Server
Enter message: How are you?
Server: Server processed: How are you?
Enter message: exit
Server: Server processed: exit
srikar@DESKTOP-0NIMS2G:~/week6$
```

```
srikar@DESKTOP-0NIMS2G: ~/week6
srikar@DESKTOP-0NIMS2G:~$ cd ~/week6
srikar@DESKTOP-0NIMS2G:~/week6$ nano client.c
srikar@DESKTOP-0NIMS2G:~/week6$ gcc client.c -o client
srikar@DESKTOP-0NIMS2G:~/week6$ ./client
Client connected to server.
Enter message:
Server: Server processed:
Enter message:
Server: Server processed:
Enter message: Hello Server
Server: Server processed: Hello Server
Enter message: How are you?
Server: Server processed: How are you?
Enter message: exit
Server: Server processed: exit
srikar@DESKTOP-0NIMS2G:~/week6$ Server started...
ing for client message...Server: command not found
srikar@DESKTOP-0NIMS2G:~/week6$ Waiting for client message...cd ~/week6
Waiting: command not found
srikar@DESKTOP-0NIMS2G:~/week6$ cd ~/week6
srikar@DESKTOP-0NIMS2G:~/week6$ ./server
Server started...
Waiting for client message...
Client: Client 1 message
Response sent to client.
Waiting for client message...
Client: ./client
Response sent to client.
Waiting for client message...
Client: Client 2 message
Response sent to client.
Waiting for client message...
```

```
srikar@DESKTOP-0NIMS2G: ~/week6
srikar@DESKTOP-0NIMS2G:~$ cd ~/week6
srikar@DESKTOP-0NIMS2G:~/week6$ ./client
Client connected to server.
Enter message: ./client
Server: Server processed: ./client
Enter message: Client 2 message
Server: Server processed: Client 2 message
Enter message:
```

Observation

- Two Named Pipes (FIFOs) were created for communication between the client and server.
- The client successfully sent messages to the server through the request FIFO.
- The server read the messages, processed them, and sent the response back to the client through the response FIFO.
- Communication was bidirectional: Client → Server and Server → Client.
- The FIFO follows the First-In-First-Out (FIFO) principle, so messages are generally read in the order in which they are written.
- When multiple clients communicate with the same server, their requests are received through the common FIFO and are handled according to the order of their writes.
- FIFO communication provides synchronization between processes because a process may wait until another process opens/writes to the FIFO.
- After the client sends `exit`, the server sends a termination response and closes the FIFOs.

Task 2: Create a POSIX signal handling program that captures SIGINT, SIGTERM, and SIGUSRI. Demonstrate asynchronous event handling and explain the role of signal handlers.

```
cd ~/week6
Client:
Response sent to client.
Waiting for client message...
Client:
Response sent to client.
Waiting for client message...
Client: Hello Server
Response sent to client.
Waiting for client message...
Client: How are you?
Response sent to client.
Waiting for client message...
Client: exit
Response sent to client.
Waiting for client message...
srikar@DESKTOP-0NIMS2G:~/week6$ cd ~/week6
srikar@DESKTOP-0NIMS2G:~/week6$ cd ~/week6
srikar@DESKTOP-0NIMS2G:~/week6$ ./client
Client connected to server.
Enter message: Client 1 message
Server: Server processed: Client 1 message
Enter message: q
Server: Server processed: q
Enter message: quit
Server: Server processed: quit
Enter message: ^C
srikar@DESKTOP-0NIMS2G:~/week6$ nano signals.c
srikar@DESKTOP-0NIMS2G:~/week6$ gcc signals.c -o signals
srikar@DESKTOP-0NIMS2G:~/week6$ ./signals
Process started. PID: 45360
Process running...
Process running...
Process running...
Process running...
Process running...
Process running...
```

Observation

- A POSIX signal handling program was successfully implemented using the `signal()` system call.
- Signal handlers were registered for `SIGINT`, `SIGTERM`, and `SIGUSR1`.
- `SIGINT` was generated using `Ctrl+C` and was successfully caught by the signal handler.
- `SIGUSR1` and `SIGTERM` were sent to the process using the `kill` command.
- Whenever a signal was received, the corresponding signal handler was executed immediately.
- The program continued its execution after handling the signals instead of terminating.
- This demonstrates asynchronous event handling, because signals can interrupt a process at any time and cause the registered handler to execute.
- The signal handler is a function that specifies the action to be performed when a particular signal is received.
- The process ID (`PID`) is required to send signals to a specific running process using the `kill` command.